

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра инженерной психологии и эргономики

Специальность «Программная инженерия»

Учебная дисциплина «Современные языки программирования»

ОТЧЕТ

по лабораторной работе №5
«Использование программного пакета Anaconda»
Вариант 22

Подготовил:

Конон С.И.

Проверила:

Семерник О.Д.

Минск 2026

Задание:

Исследование зависимости между временем выполнения операций и размером данных: Создайте Jupyter Notebook, в котором создаются массивы от 10^4 до 10^7 элементов. Измерьте время выполнения базовых операций (+, *, log, exp) в numpy. Постройте графики зависимости времени от размера массива. Сравните с теми же операциями, выполненными над списками Python. Представьте выводы в виде markdown-анализов и графиков.

Также для каждого варианта следует выполнить следующие требования:

- Установить и настроить Anaconda
- Создать виртуальную среду lab5env с Python 3.10
- Установить: numpy, pandas, matplotlib, seaborn, openpyxl
- Запуск и ведение Jupyter Notebook (все действия и объяснения фиксируются в блокноте)

Для того, чтобы настроить Anaconda, введем в терминале команду `conda create -n lab5env python=3.10`. Наш lab5env уже будет включать в себя такие библиотеки как numpy, pandas, matplotlib, seaborn, openpyxl.

Чтобы создать Jupyter Notebook, следует нажать правой кнопкой мыши на папку, где требуется создать Jupyter Notebook.

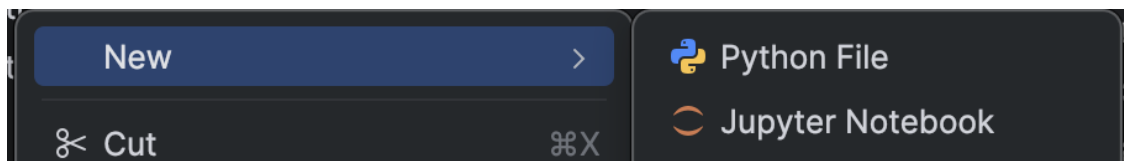


Рисунок 1 – Создание Jupyter Notebook

Для данной лабораторной работы нам понадобятся следующие библиотеки:

- math;
- numpy;
- time;
- matplotlib.

```
from math import log, exp
import numpy as np
import time
import matplotlib.pyplot as plt
```

Рисунок 2 – Демонстрация импорта библиотек

Так как нам следует работать с массивами данных, размеры которых равны от 10^4 до 10^7 элементов, – для удобства создадим отдельный размер с массивами, а также со знаками операций:

```
sizes = [10**4, 10**5, 10**6, 10**7]

op = ["+", "*", "log", "exp"]
```

Рисунок 3 – Демонстрация инициализации массивов

Для нашего удобства и лаконичности кода, создадим словари с результатами обычных списков python и списков numpy. Воспользуемся для этого dict comprehensions.

```
results_np = {i: [] for i in op}
results_py = {i: [] for i in op}
```

Рисунок 4 – Генерация словарей с помощью dict comprehensions

Теперь перейдем к основной части, а именно к сравнению времени выполнения наших операций, которые перечислены в списке `op`. Демонстрация кода основной части предоставлена ниже.

Листинг кода основной части:

for i in sizes:

np_arr = np.arange(1, i + 1)

py_list = [j for j in range(1, i+1)]

#numpy

start = time.time()

_ = np_arr + 1

results_np["+"].append(time.time() - start)

start = time.time()

_ = np_arr * 4

results_np["*"].append(time.time() - start)

start = time.time()

_ = np.log(np_arr)

results_np["log"].append(time.time() - start)

start = time.time()

```
_ = np.exp(np_arr / i)
results_np["exp"].append(time.time() - start)
```

```
#python list
start = time.time()
_ = [x + 2 for x in py_list]
results_py["+"].append(time.time() - start)
```

```
start = time.time()
_ = [x * 4 for x in py_list]
results_py["*"].append(time.time() - start)
```

```
start = time.time()
_ = [log(x) for x in py_list]
results_py["log"].append(time.time() - start)
```

```
start = time.time()
_ = [exp(x / i) for x in py_list]
results_py["exp"].append(time.time() - start)
```

При помощи `numpy` мы можем не создавать циклы, чтобы сделать какое-либо действие с каждым элементом списка.

Допустим, посмотрим на первую операцию добавления к каждому элемента списка единицы. Сравним как выглядит это при помощи чистого `python` и библиотеки `numpy`.

Пример кода на чистом `python`:

```
_ = [x + 2 for x in py_list]
```

Пример кода с использованием библиотеки `numpy`:

```
_ = np_arr + 1
```

Так как нам нигде не нужно будет в будущем использовать эти значения – хорошим вариантом будет использовать вместо бессмысленного названия переменной, создать переменную `_`.

В каждом из случаев работа с `numpy` будет выглядеть более читаемо.

При использовании функции `exp()` что в `numpy`, что и в чистом `python`, я делил каждый элемент списка на размер нашего списка, то есть нормировал. В ином случае, нам бы в консоль приходила бы ошибка.

Рассмотрим часть с построением графиков с помощью библиотеки `matplotlib`.

Листинг части кода с построением графиков с помощью библиотеки `matplotlib`:

for operation in op:

```
plt.plot(sizes, results_np[operation], label='NumPy')
plt.plot(sizes, results_py[operation], label='Python list')
```

```
plt.title(f"Операция {operation}")
plt.xlabel("Размер массива")
plt.ylabel("Секунды")

plt.legend()

plt.show()
```

После выполнения основной части программы были получены графики зависимости времени выполнения операций от размера массива. Для каждой операции строится отдельный график, на котором сравнивается время выполнения для библиотеки `numpy` и стандартных списков `python`.

На графиках можно наблюдать, как изменяется время выполнения операций при увеличении размера массива от 10^4 до 10^7 элементов.

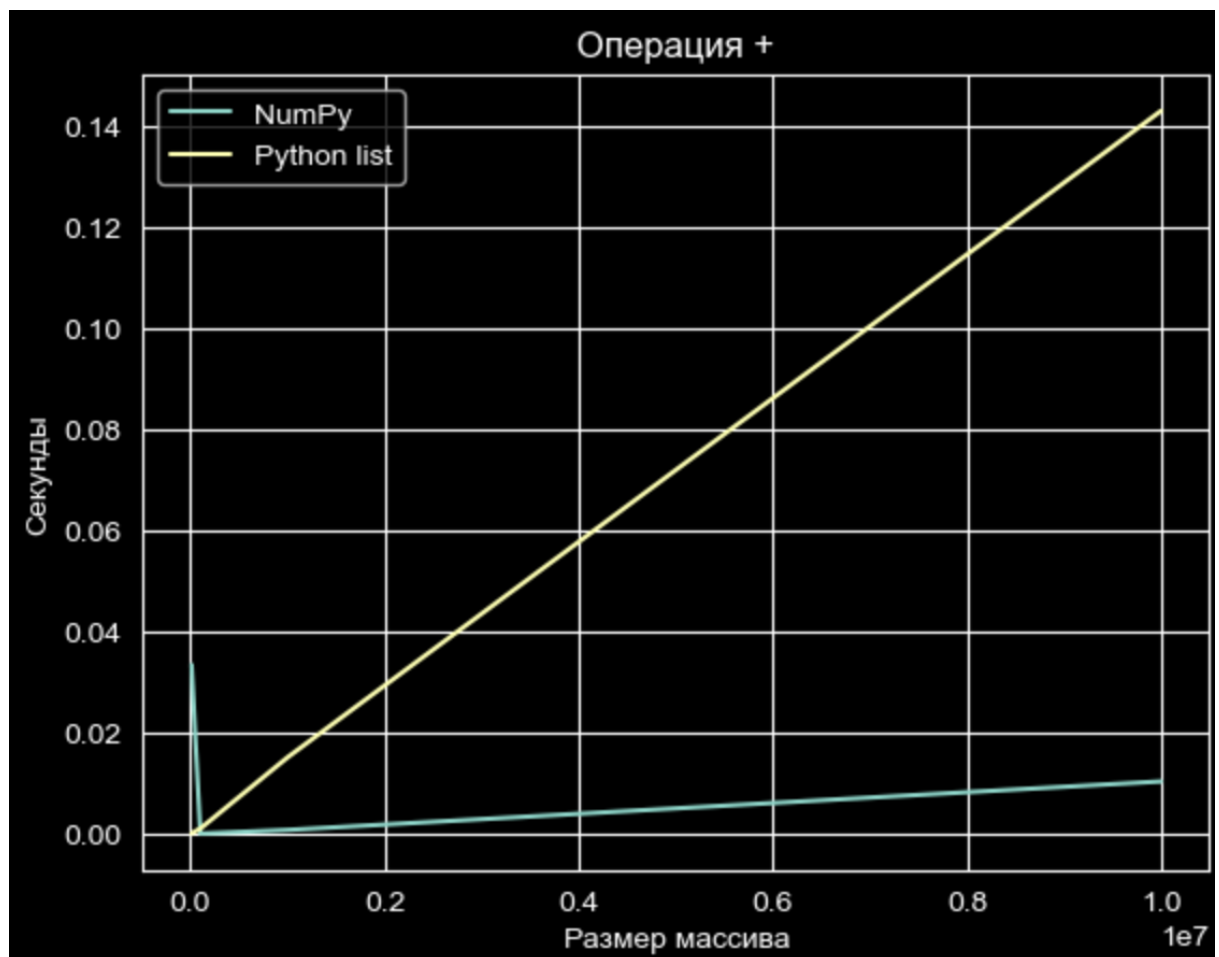


Рисунок 5 – График зависимости времени выполнения операции сложения от размера массива

На данном графике видно, что операция сложения выполняется сильно быстрее при использовании библиотеки `numpy`. Это связано с тем, что `numpy` использует оптимизированные низкоуровневые реализации на языке C и

применяет векторизованные операции, позволяющие выполнять действия сразу над всем массивом.

В случае использования стандартных списков python операция выполняется значительно медленнее, так как требуется перебор каждого элемента списка с помощью цикла.

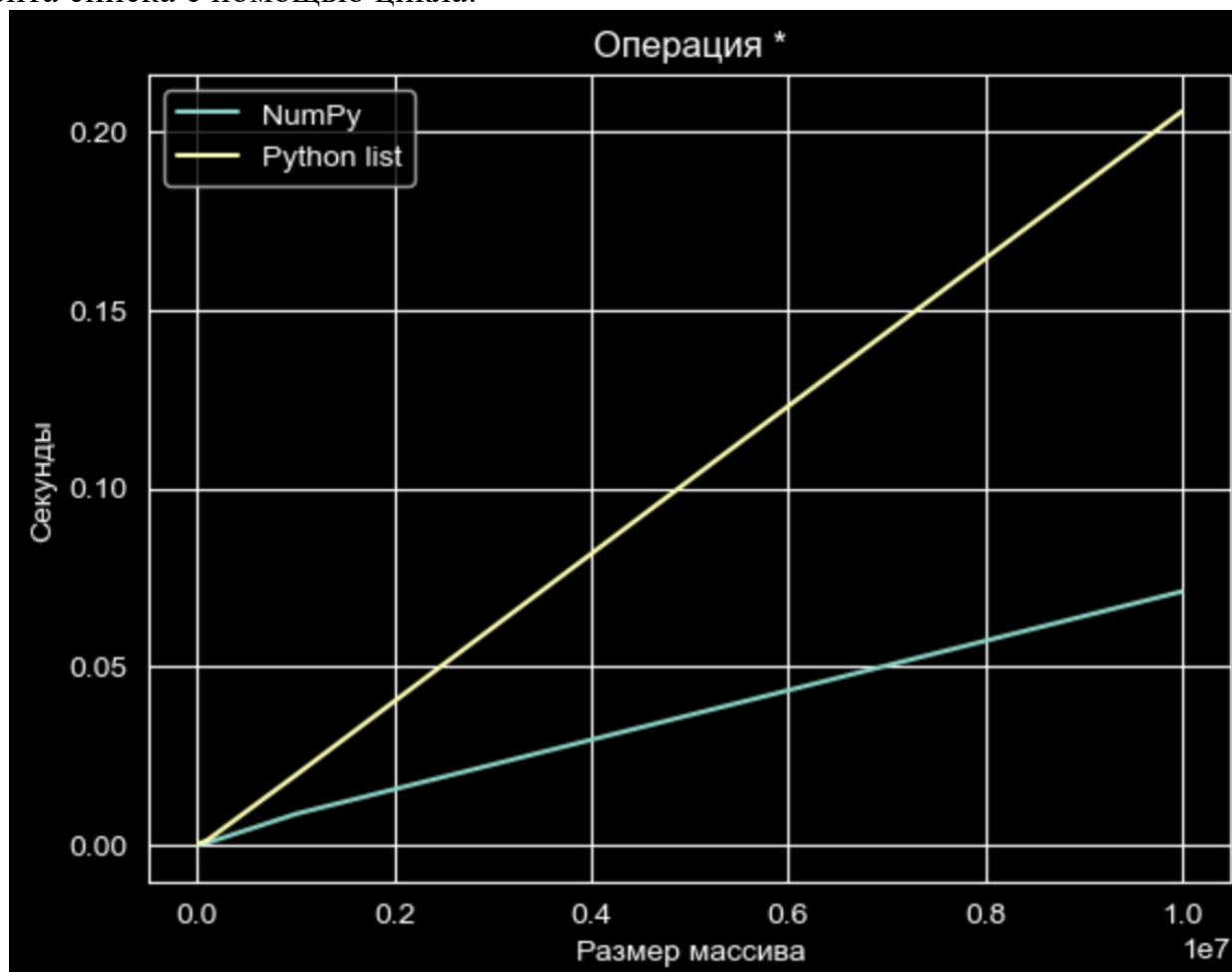


Рисунок 6 – График зависимости времени выполнения операции умножения от размера массива

Похожая ситуация наблюдается и при выполнении операции умножения. Numpy выполняет операцию над всем массивом одновременно, тогда как при использовании списков python требуется обработка каждого элемента отдельно.

Стоит отметить, что при увеличении размера массива время выполнения операций для списков python растет значительно быстрее.

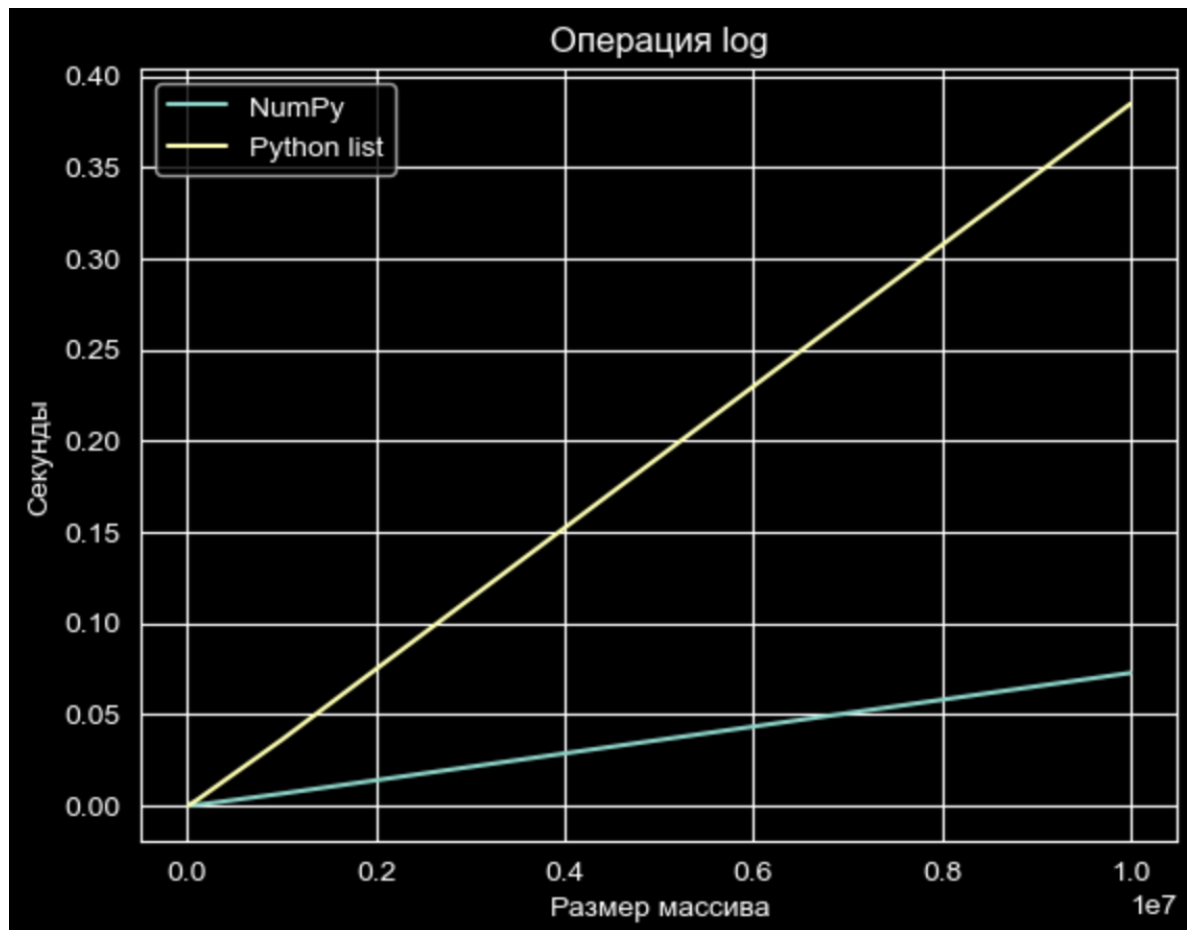


Рисунок 7 – График зависимости времени выполнения операции `log` от размера массива

При выполнении функции `log()` также наблюдается преимущество использования `numpy`. В библиотеке `numpy` функция применяется сразу ко всему массиву, тогда как в чистом `python` она вызывается для каждого элемента списка отдельно, что приводит к увеличению времени выполнения.

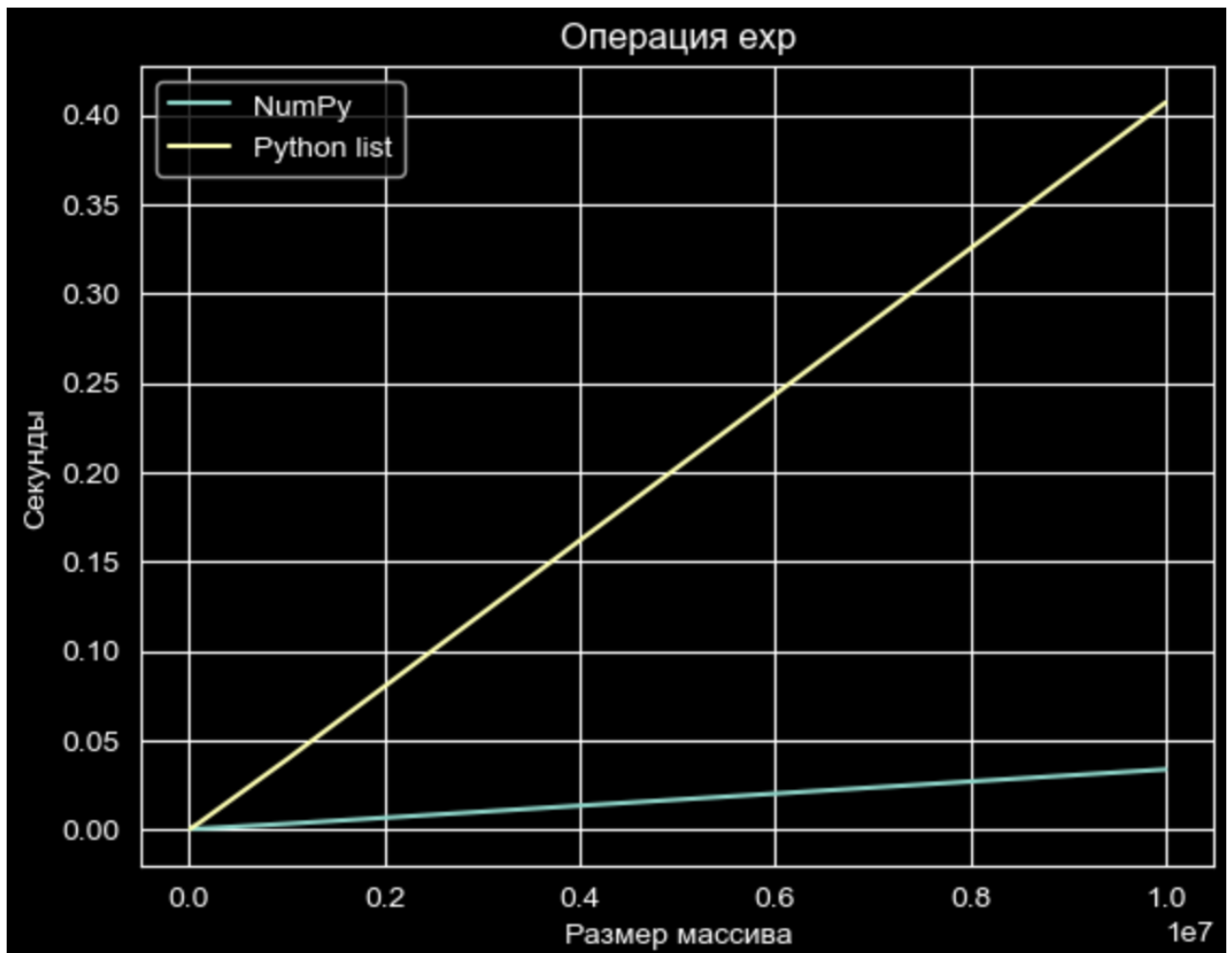


Рисунок 8 – График зависимости времени выполнения операции `exp` от размера массива

При выполнении функции `exp()` ситуация аналогична предыдущим операциям. NumPy показывает значительно более высокую производительность по сравнению со стандартными списками python.

Кроме того, при увеличении размера массива разница во времени выполнения становится всё более заметной.

Вывод:

В ходе выполнения лабораторной работы было проведено исследование зависимости времени выполнения различных операций от размера массива данных. Для этого были созданы массивы размером от 10^4 до 10^7 элементов, после чего измерялось время выполнения операций сложения, умножения, а также функций `log` и `exp`. Сравнение проводилось между стандартными списками python и массивами библиотеки numpy. По полученным данным были построены графики, позволяющие наглядно увидеть изменение времени выполнения операций при увеличении объема данных.

В результате было получено, что операции, выполненные с использованием библиотеки `numpy`, выполняются значительно быстрее, чем аналогичные операции над стандартными списками `python`.

Разница во времени выполнения становится более заметной по мере увеличения размера массива. Это связано с тем, что `numpy` использует векторизованные операции и оптимизированные низкоуровневые реализации, благодаря чему действия выполняются сразу над всем массивом данных.

При использовании стандартных списков `python` операции выполняются путем последовательного перебора элементов, что приводит к увеличению времени выполнения программы, особенно при работе с большими массивами данных.

При обработке больших объемов числовой информации использование библиотеки `numpy` является более эффективным и позволяет значительно повысить производительность вычислений.

Контрольные вопросы:

1. Anaconda – это дистрибутив Python для анализа данных и разработки, который содержит Python, менеджер пакетов и готовые библиотеки.
2. `conda` – менеджер пакетов и окружений для разных языков, а `pip` – менеджер пакетов только для Python.
3. В Anaconda входят: Python, `conda`, Anaconda Navigator, Jupyter Notebook, Spyder и множество библиотек для анализа данных.
4. Виртуальное окружение – изолированная среда с собственной версией Python и установленными пакетами.
5. Создание: `conda create -n имя_среды python`
Активация: `conda activate имя_среды`
6. Установка: `conda install пакет`
Обновление: `conda update пакет`
7. `conda-forge` – это общественный репозиторий пакетов для `conda`.
8. Сохранение окружения: `conda env export > environment.yml`
9. Импорт: `conda env create -f environment.yml`
10. Anaconda решает проблемы установки библиотек, зависимостей и управления версиями.
11. Anaconda Navigator позволяет управлять окружениями, пакетами и запускать приложения (Jupyter, Spyder и др.) через графический интерфейс.
12. Jupyter Notebook состоит из ячеек (`cells`), которые выполняются по отдельности.
13. Типы ячеек: Code, Markdown, Raw.
14. Интерактивность, выполнение кода по частям, удобство для анализа данных и визуализации.
15. Запуск из консоли: `jupyter notebook`
16. Да, `pip` можно использовать внутри среды `conda`.
17. Нужно активировать среду и установить пакет: `pip install пакет`.
18. Возможны конфликты версий и зависимостей пакета.

19. `environment.yml` – файл с описанием среды и списком всех пакетов и их версий.
20. Переключение между средами: `conda activate имя_среды`.